

דוח מסכם: פרויקט אריזה וניתוח תעבורת TCP/IP

שם הקורס: רשתות תקשורת מחשבים

מרצה: אפי פרופוס

שמות הסטודנטים: להב סרוסי וגיתם שמעוני

תאריך: 19/01/2026

חלק 1: אריזה ולכידת המנות (Encapsulation & Capture)

יצירת קובץ הנתונים (CSV Generation)

בשלב הראשון הוכן באופן ידני קובץ קלט בפורמט CSV אשר נועד לדמות את המידע הנוצר בשכבת היישום (Application Layer). יצירה ידנית זו אפשרה שליטה מלאה בתוכן ההודעות ובפורמט שלהן לצורך זיהוי ואימות קל בשלבים הבאים. הקובץ נבנה בהתאם לדרישות הפרויקט והסקריפט וכלל את העמודות הבאות:

- **msg_id** - מזהה ייחודי להודעה.
- **app_protocol** – פרוטוקול היישום (לדוגמא HTTP)
- **src_app** - יישום שולח (Source).
- **dst_app** - יישום יעד (Destination).
- **message** - מחרוזת טקסט המהווה את Payload (תוכן ההודעה).
- **timestamp** - חותמת זמן לתיעוד.
- **src_port** - פורט המקור שנבחר אקראית בקוד כמספר רנדומלי (בין 1024 ל-65535).
- **dst_port** - פורט היעד נקבע כפורט ייחודי (למשל 12345) לסינון קל ב-Wireshark.

תהליך האריזה ועיבוד הנתונים (Encapsulation Process)

תהליך האריזה בוצע באמצעות הרצת קוד Python בסביבת Jupyter Notebook. הקוד מדמה את המנגנון בו מערכת ההפעלה עוטפת נתונים לפני שליחתם ברשת. השתמשנו בספריית scapy המאפשרת ליצור מנות (Packets) מאפס, ובספריית pandas המאפשרת לעבוד עם קבצי CSV בצורה נוחה:

1. **קריאה וטעינה:** הסקריפט טען את קובץ ה-CSV בעזרת ספריית pandas וביצע בדיקת תקינות (Validation) בה ווידאנו שאכן הקובץ מכיל את העמודות הנדרשות.
2. **יצירת שכבת התעבורה (Transport Layer):**
 - לכל הודעה נוצרה כותרת TCP Header ייחודי.
 - הוגדרו פורט המקור ופורט היעד.
 - הוגדרו דגלים (Flags) כגון PSH ו-Ack לציון העברת מידע.
3. **יצירת שכבת הרשת (Network Layer):**
 - ה-TCP Header נעטפה בתוך IP Header.
 - כתובות המקור והיעד הוגדרו כ-127.0.0.1 (Localhost Loopback).
 - שדה הפרוטוקול הוגדר כ-6 (TCP).

4. הרכבת המנה (Payload Assembly): המידע משכבת האפליקציה (הודעת הטקסט) הוצמד ל-IP Header, ליצירת מבנה היררכי מלא: Application -> TCP Header -> IP Header Data.

מתודולוגיית לכידת התעבורה (Capture Methodology)

תהליך הלכידה בוצע בזמן אמת באמצעות כלי ה-Wireshark, במקביל להרצת הקוד ב-Jupyter:

- **בחירת הממשק:** נבחר מתאם ה-Loopback (Adapter for loopback traffic capture) כיוון שהתקשורת היא פנימית במחשב.
- **סנכרון:** ההקלטה (Capture) הופעלה לפני הרצת פקודת השליחה בקוד, כדי להבטיח את תפיסת כל המנות.
- **סינון:** לאחר סיום השליחה ועצירת ההקלטה, הופעל מסנן תצוגה על פורט היעד הייחודי שהוגדר (ip.addr == 127.0.0.1 && tcp.port == 12345) כדי לבודד את המנות הרלוונטיות לפרויקט מתוך כלל תעבורת הרקע של מערכת ההפעלה.

ניתוח הממצאים ב-Wireshark

מצורף לפרויקט ניתוח של המנות שנלכדו בקובץ jupyter_pickup1.pcapng, המאמת את הצלחת תהליך האריזה.

בתמונה להלן ניתן לראות את רצף המנות שנשלחו מהסקריפט וסוננו לפי פורט היעד (למשל 12345). ניתן לזהות בבירור את כתובות ה-127.0.0.1 Loopback ככתובת המקור והיעד ואת פרוטוקול ה-TCP.

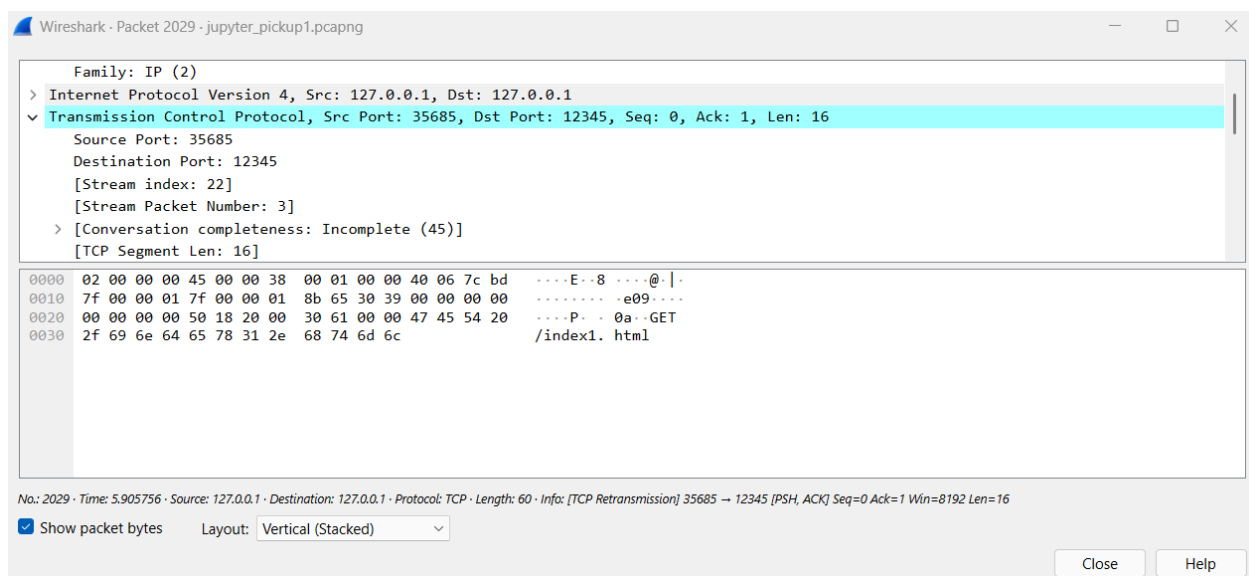
1. רשימת המנות אחרי פילטור:

No.	Time	Source	Destination	Protocol	Length	Info
1738	2.839142	127.0.0.1	127.0.0.1	TCP	58	35685 → 12345 [SYN] Seq=0 Win=8192 Len=14
1740	2.839419	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1963	3.848858	127.0.0.1	127.0.0.1	TCP	58	[TCP Port numbers reused] 35685 → 12345 [SYN] Seq=0 Win=8192 Len=14
1964	3.849136	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
1965	4.855998	127.0.0.1	127.0.0.1	TCP	58	[TCP Port numbers reused] 35685 → 12345 [SYN] Seq=0 Win=8192 Len=14
1966	4.856254	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2029	5.905756	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2030	5.906026	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2033	6.015702	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2034	6.015897	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2035	6.158704	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2036	6.158969	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2037	6.287259	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2038	6.287512	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2039	6.396370	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2040	6.396587	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2045	6.506369	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2046	6.506553	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2055	6.648585	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16
2056	6.648813	127.0.0.1	127.0.0.1	TCP	44	12345 → 35685 [RST] Seq=1 Win=0 Len=0
2057	6.755010	127.0.0.1	127.0.0.1	TCP	60	[TCP Retransmission] 35685 → 12345 [PSH, ACK] Seq=0 Ack=1 Win=8192 Len=16

2. מבנה ה-Headers: ניתוח מעמיק של מנה בודדת חושף את השכבות שהוגדרו בקוד:

1. **Internet Protocol (IPv4):** מציג את ה-Header Length, ה-Header, ה-TTL, ואת ה-Source Address.

2. **Transmission Control Protocol (TCP):** מציג בין היתר את פורט המקור והיעד, ואת מספרי ה-Sequence וה-Acknowledgment.



3. אימות תוכן ההודעה (Payload Verification): בתצוגת ה-Hex Dump המופיעה למטה, בחלק הקריא המופיע מימין, ניתן לראות את תוכן ההודעה המקורי כפי שהוזן בקובץ ה-CSV. עובדה זו מאשרת כי המידע עבר את כל שכבות הפרוטוקול, נארז, נשלח, ונלכד ללא שיבוש.

חלק 2: כתיבת יישום רשת וניתוח תעבורה (Application and Traffic Analysis)

הסבר כללי על המערכת ומבנה הקוד

בחלק זה של הפרויקט, תוכננה ומומשה מערכת צ'אט מבוססת ארכיטקטורת שרת-לקוח (Client-Server) הפועלת על פרוטוקול TCP. המערכת מאפשרת למספר משתמשים להתחבר במקביל ולנהל שיחות פרטיות בזמן אמת.

מבנה המערכת:

- **השרת (Server):** מהווה את הליבה של המערכת. הוא מאזין לפורט קבוע (10000), מנהל את רשימת המחברים ומנתב הודעות בין לקוחות. השרת משתמש ב-Threading (ריבוי תהליכים) כדי לטפל בכל לקוח נפרד, מה שמאפשר טיפול ב-5 לקוחות ויותר בו-זמנית ללא חסימת התקשורת.
- **הלקוח (Client):** אפליקציית הקצה המאפשרת למשתמש להזין שם משתמש, להתחבר לשרת ולשלוח הודעות בפורמט ייעודי (Target:Message). הלקוח מריץ תהליכון נפרד להאזנה להודעות נכנסות (Receive Thread), כך שניתן לקבל הודעות גם בזמן שהמשתמש מקליד הודעה חדשה.
- **ניהול הנתונים (Database Structure):** המערכת מנהלת את הנתונים בזיכרון (In-Memory) באמצעות מילון (Python Dictionary), הממפה בין שם המשתמש (Key) לבין אובייקט ה-Socket הפעיל שלו (Value).

הוראות התקנה והרצה (ניתן גם לראות בקובץ ה README המושקע המצורף לפרויקט)

לחצו עליו

המערכת נכתבה בשפת Python ועושה שימוש בספריות סטנדרטיות בלבד (socket, threading), ולכן אינה דורשת התקנות חיצוניות.

תהליך ההרצה:

1. **הפעלת השרת:** הרצת הקובץ server.py שבתוך תיקיית backend. השרת ידפיס את כתובת ה-IP המקומית וימתין לחיבורים.
2. **הגדרת הלקוח:** עדכון משתנה HOST בקובץ client.py שבתוך תיקיית backend לכתובת ה-IP של השרת.
3. **הפעלת הלקוח:** הרצת הקובץ client.py בטרמינל נפרד ובכמה מחשבים שונים על אותה הרשת (ניתן להריץ מספר מופעים עבור מספר לקוחות).
4. **שימוש:** הזנת שם משתמש ושליחת הודעות בפורמט Username:Message. שימוש נוסף - שליחה הודעה לכלל חברי הציאט בפורמט everyone:Message

דוגמאות קלט ופלט

במהלך הבדיקה בוצע תרחיש שיחה בין שני משתמשים: Gitam ו-Lahav אשר נמצאים בשני מחשבים שונים על אותה הרשת:

תצוגת הקלט-פלט מהצד של servern:

```
Run  server x  client x
C:\Users\lahav\Projects\pythonProjects\workNetworks\ComputerNetworksProj
[STARTING] Server is starting...
[LISTENING] Server is listening on 0.0.0.0:10000
--> Tell the client to connect to IP: 192.168.0.11
[ACTIVE CONNECTIONS] 1
[NEW CONNECTION] ('192.168.0.11', 50295) connected.
[REGISTERED] User 'Lahav' added from ('192.168.0.11', 50295)
[ACTIVE CONNECTIONS] 2
[NEW CONNECTION] ('192.168.0.180', 60056) connected.
[REGISTERED] User 'Gitam' added from ('192.168.0.180', 60056)
[Lahav]: Gitam:Hey, how are you?
[Gitam]: Lahav:I'm fine, and you?
[Lahav]: Gitam:I feel great. Do you think we'll succeed in the project?
[Gitam]: Lahav:I'm sure we will, don't worry!
[ERROR] Client ('192.168.0.11', 50295) disconnected abruptly.
[DISCONNECT] User 'Lahav' removed.
[ERROR] Client ('192.168.0.180', 60056) disconnected abruptly.
[DISCONNECT] User 'Gitam' removed.

Process finished with exit code -1
```

תצוגת הקלט-פלט מהצד של client בשם Lahav:

```
Run  server x client x
C:\Users\lahav\Projects\pythonProjects\workNetworks\ComputerNetworksProject\.venv
Attempting to connect to 192.168.0.11:10000...
Enter your username: Lahav

Welcome Lahav! To chat, type: TargetName:Message
Your message: Your message (format: User:Msg or 'exit'): Gitam:Hey, how are you?
Your message (format: User:Msg or 'exit'):
[Gitam]: I'm fine, and you?
Your message: Gitam:I feel great. Do you think we'll succeed in the project?
Your message (format: User:Msg or 'exit'):
[Gitam]: I'm sure we will, don't worry!
Your message: exit
Client closed.

Process finished with exit code 0
```

תצוגת הקלט-פלט מהצד של client בשם Gitam:

```
Run  script x
C:\Users\mual\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users
Attempting to connect to 192.168.0.11:10000...
Enter your username: Gitam

Your message (format: User:Msg or 'exit'):
Welcome Gitam! To chat, type: TargetName:Message
Your message:
[Lahav]: Hey, how are you?
Your message: Lahav:I'm fine, and you?
Your message (format: User:Msg or 'exit'):
[Lahav]: I feel great. Do you think we'll succeed in the project?
Your message: Lahav:I'm sure we will, don't worry!
Your message (format: User:Msg or 'exit'): exit
Client closed.

Process finished with exit code 0
```

ניתוח תעבורת היישום (Traffic Analysis)

לכידת התעבורה בוצעה באמצעות Wireshark בזמן הרצת היישום. בקובץ הלכידה part_two_pickup.pcapng ניתן לראות את מהלך השיחה המלא. להלן ניתוח המנות לפי מודל ה-TCP/IP:

א. יצירת החיבור (Connection Establishment - 3-Way Handshake) בתחילת התקשורת, ניתן לראות את תהליך לחיצת היד המשולשת בין הלקוח לשרת בפורט 10000.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.0.11	192.168.0.11	TCP	56	50295 → 10000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000060	192.168.0.11	192.168.0.11	TCP	56	10000 → 50295 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3	0.000077	192.168.0.11	192.168.0.11	TCP	44	50295 → 10000 [ACK] Seq=1 Ack=1 Win=65280 Len=0

על ידי ה-3 שורות הראשונות של Handshaken ניתן לראות שהלקוח שולח בקשת SYN, השרת משיב ב-SYN, ACK והלקוח מאשר ב-ACK. זהו הבסיס לחיבור אמין.

ב. שכבת הרשת (Network Layer - IPv4)

- כתובות: בלכידה זו (שבוצעה בסביבה מקומית), ניתן לראות שכתובת המקור וכתובת היעד הן 127.0.0.1 (Loopback Interface).

- פרוטוקול: שדה הפרוטוקול מצביע על TCP, ערך 6.

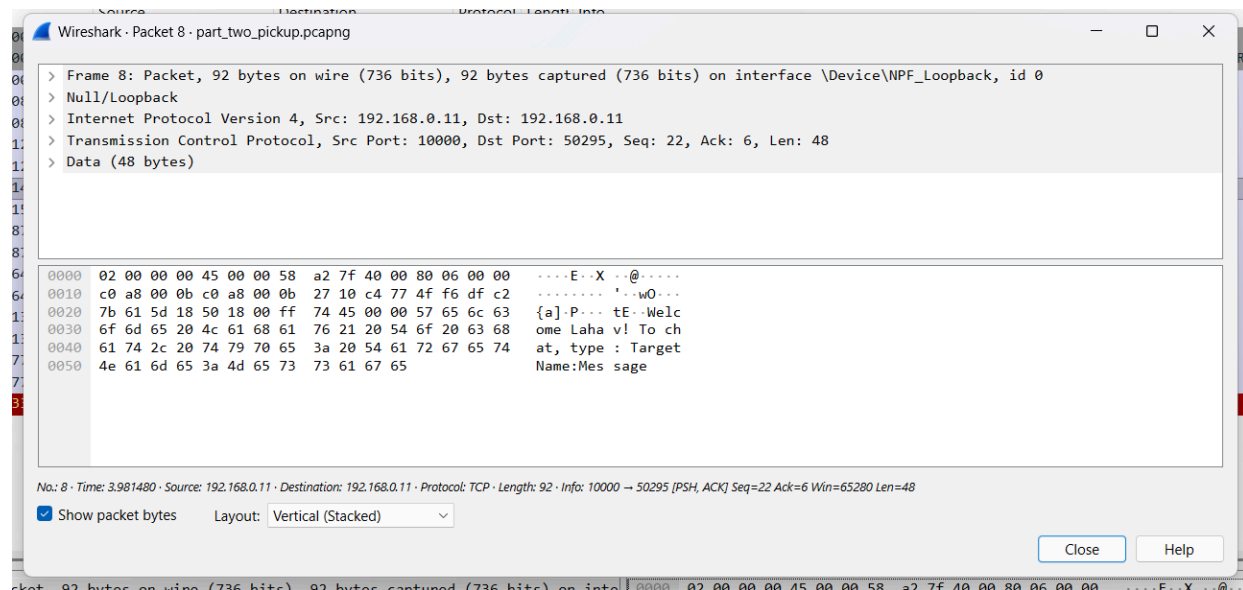
ג. שכבת התעבורה (Transport Layer - TCP)

- פורטים: ניתן לראות שהתקשורת מתבצעת מול פורט היעד 10000 (הפורט שהוגדר בשרת) ומפורט מקור אקראי (לדוגמה 35685 בלכידה הנוכחית).

- דגלים (Flags): הודעות המכילות מידע (Data) מסומנות בדגל (PSH (Push, המורה למקבל להעביר את המידע מיידית לאפליקציה, ובדגל ACK לאישור קבלה.

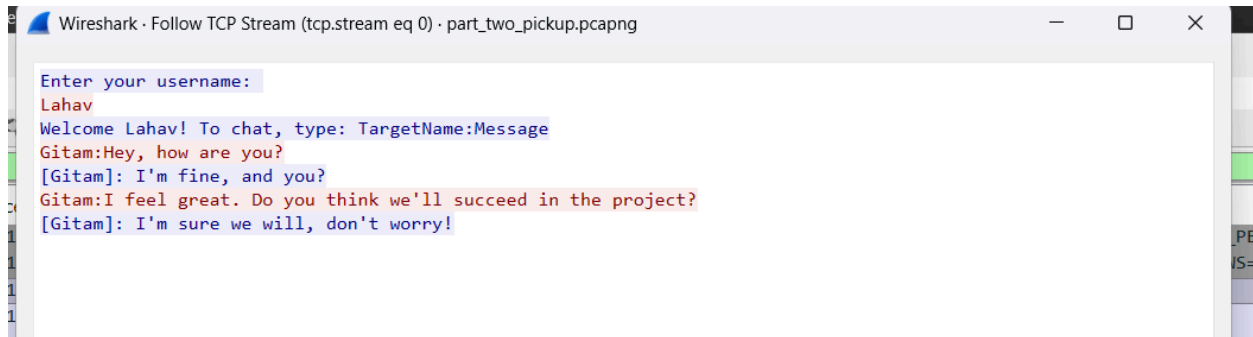
ד. שכבת היישום והתוכן (Application Layer Payload)

ניתן לראות את תוכן ההודעות בבירור בגוף החבילה.



בחלונית המידע (Payload) ניתן לראות את המחרוזת שנשלחה:

Welcome Lahav! To chat, type: TargetName:Message
הדבר מאמת שהמידע נארז, נשלח, ופוענח בהצלחה על ידי מערכת ההפעלה והאפליקציה.



מזורף שימוש בכלי Follow Stream מציג את רצף השיחה המלא כפי שתועד ברשת: הזנת השמות והעברת ההודעות ביניהם.

את ההודעות שלכדנו, העברנו לקובץ "chat_traffic_analysis.csv". להלן תיאור קובץ הנתונים:

קובץ הנתונים מהווה תצוגה טבלאית ומעובדת של תעבורת הרשת שנלכדה. הקובץ מרכז את המידע הטכני משכבות ה-TCP/IP השונות עבור כל חבילה הרלוונטית לפורט השירות (10000).

להלן פירוט העמודות בקובץ ומשמעותן הטכנית:

1. **Timestamp (חותמת זמן):** מציג את הזמן המדויק בו כל חבילה נקלטה ברשת. כך ניתן לנתח את סדר האירועים הכרונולוגי (Sequence) – החל מיצירת הקשר (Handshake) ועד להעברת ההודעות וסגירת החיבור.
2. **Source IP & Destination IP (כתובות מקור ויעד):** מייצג את שכבת הרשת (Layer 3). בקובץ זה, הכתובות המופיעות הן 127.0.0.1 (Loopback), המעידות כי הלקוח והשרת תקשרו בתוך אותו ה-Host במהלך הסימולציה. חשוב לציין ששני הלקוחות תקשרו ממחשבים שונים.
3. **Source Port & Destination Port (פורטים):** מייצג את שכבת התעבורה (Layer 4). ניתן לראות בבירור את פורט השרת הקבוע (10000) מול פורטים דינמיים שהוקצו לצד הלקוח ע"י מערכת ההפעלה (במקרה זה הפורט הדינמי הוא 50292). עמודות אלו מאמתות שהתעבורה נותבה ליישום הצ'אט הספציפי שפותח.
4. **TCP Flags (דגלי בקרה):** מציג את הסטטוס של כל חבילה בפרוטוקול TCP.
5. **Message Content (תוכן ההודעה):** מייצג את שכבת היישום (Layer 7). עמודה זו מכילה את המידע הגולמי שעבר ברשת. בקובץ זה ניתן לקרוא את הטקסט המפורש של השיחה, מה שמוכיח שהמידע נארז ונשלח בהצלחה ללא שיבושים או הצפנה שמנעה את קריאתו.

שימוש ב-Al -

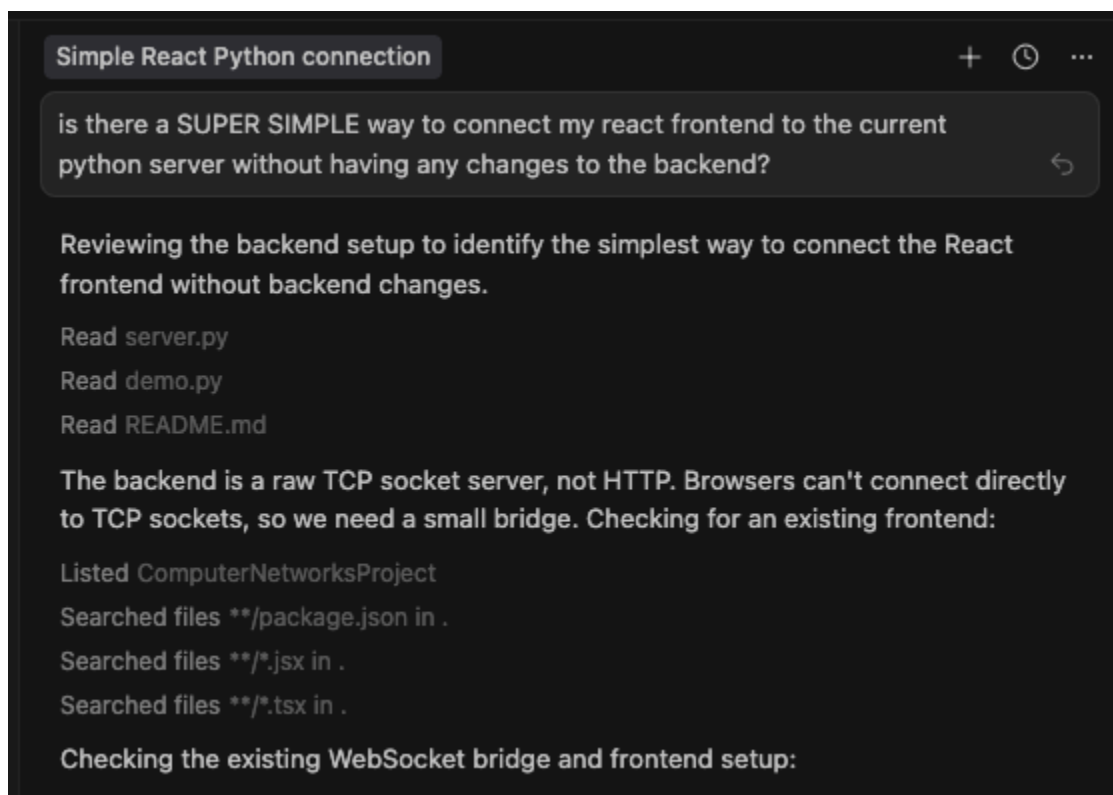
במהלך העבודה השתמשנו ב-Al בשלל דרכים.
ראשית, עבור **התכנון** הראשוני - איך לגשת לפרויקט, מה תהיה הדרך הנכונה ביותר לממש את דרישותיו.

עבור **הבנייה** - נעזרנו בכלים כמו Gemini ו Cursor בשביל לפתח עבורנו תוספות בבאקאנד (צד שרת).
Demon לדוגמה, רעיון משותף שעלה לנו ומימשנו אותו בקלות בעזרת Cursor.
עבור תכנון Bridge שמחבר בין צד הלקוח לצד השרת בעזרת Websockets, נעזרנו ב-Gemini.

עבור ייצור קומפוננטות UI פשוטות בפרונט, השתמשנו ב-Cursor בשביל לזרז את העבודה.

עבור קובץ Readmen בגיטהאב (שממש לא כדאי לפספס! הוא ברמה גבוהה מאוד!) נעזרנו ב-Gemini
בשביל לסגן את הטקסט כקובץ .md שמתאים לגיטהאב, ולתרגם אותו מאנגלית לעברית.

דוגמאות לפרומפטים:



```
Simple React Python connection + ⌚ ...

is there a SUPER SIMPLE way to connect my react frontend to the current
python server without having any changes to the backend? ↩

Reviewing the backend setup to identify the simplest way to connect the React
frontend without backend changes.

Read server.py
Read demo.py
Read README.md

The backend is a raw TCP socket server, not HTTP. Browsers can't connect directly
to TCP sockets, so we need a small bridge. Checking for an existing frontend:

Listed ComputerNetworksProject
Searched files **/package.json in .
Searched files **/*.jsx in .
Searched files **/*.tsx in .

Checking the existing WebSocket bridge and frontend setup:
```

i am gonna send you my old hebrew README, and the
..new updated english readme
Update the hebrew readme to match the new ENGLISH
version

TCP Chat System #

Gitam and Lahav present:** A Computer Networking**
class project - A real-time chat application with TCP
..backend and React frontend

Quick Start ##

Install Dependencies .1 ###

```
bash``  
cd backend  
pip install -r requirements.txt  
...
```

Start the Server .2 ###

Frontend color scheme readability

⚙️ LoginPage.tsx Change the entire color scheme all around the frontend. It looks terrible and really hard to see what is written.

Reviewing the current color scheme across the frontend, starting with LoginPage and related styling files.

Read LoginPage.tsx
Read index.css
Read App.css
Read ChatPage.tsx
Read App.tsx
Read tailwind.config.js

Checking other component files for styling:

Read MessageList.tsx
Read MessageInput.tsx
Read UserList.tsx
Read ConnectionStatus.tsx

The dark gradient background conflicts with light theme colors, causing poor contrast. Updating the color scheme for better readability with a proper dark theme and improved contrast.

Simple TCP server implementation review

+ ⌚ ...

Look at my backend implementation, is this right? i should only handle a simple tcp connection server that moves messages between clients. I feel like its over complicated now. What do you think ?

↩

Reviewing the backend implementation to assess complexity. Examining the key files.

Read `server.py`

Read `websocket_bridge.py`

Read `server.py`

Read `tcp_client.py`

Read `message_processor.py`

Read `websocket_handler.py`

Read `config.py`

Checking the README to understand the project requirements:

Read `README.md`

Analysis

TCP server (`server.py`) — simple and correct

- Multi-threaded TCP server
- Handles client registration
- Forwards messages between clients
- Clean and straightforward

WebSocket bridge — more complex than needed

The bridge adds complexity:

- Multiple files (`tcp_client.py`, `message_processor.py`, `websocket_handler.py`, `config.py`)
- Mixing `async/await` (WebSocket) with `threading` (TCP client)
- Message parsing/translation layer

Recommendation

If you only need a simple TCP server that moves messages between clients, you can remove the WebSocket bridge. The TCP server in `server.py` is sufficient.

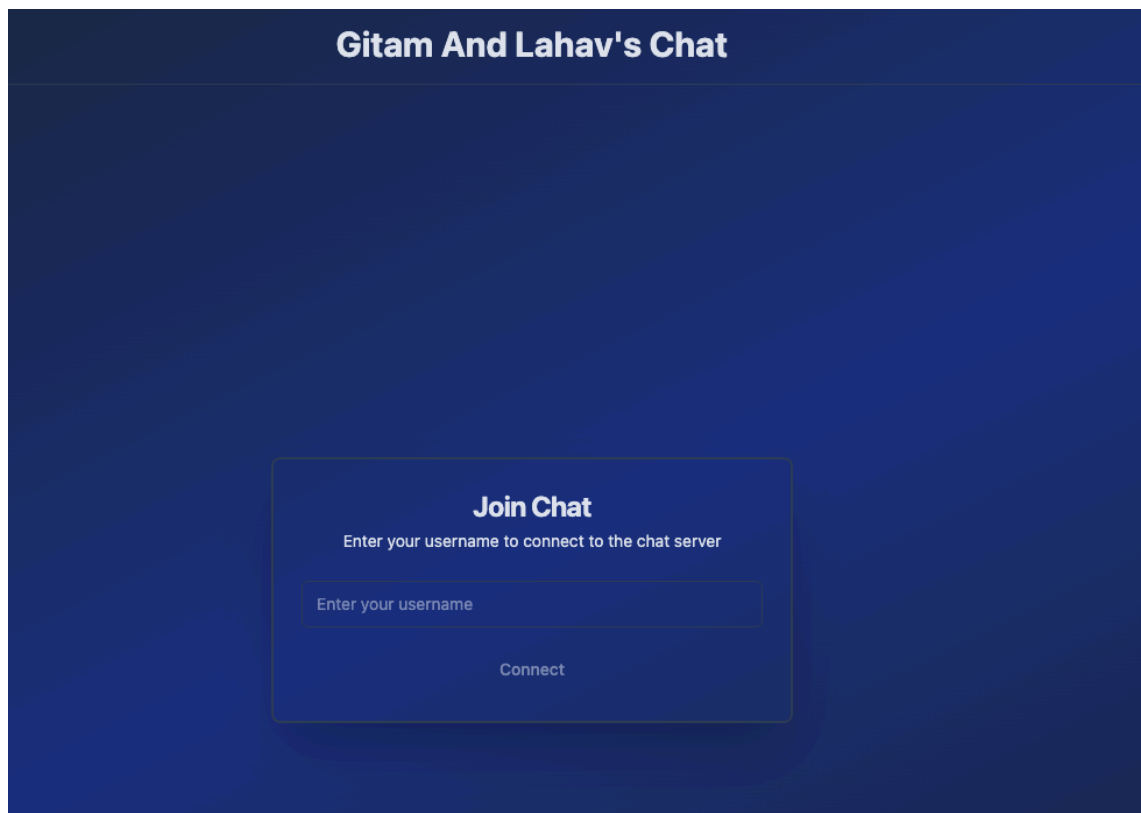
If you need the web frontend, keep the bridge but simplify it:

1. Combine some files (e.g., merge `tcp_client.py` and `message_processor.py`)
2. Use `async` throughout (replace `threading` with `asyncio` for TCP)
3. Reduce message parsing complexity

Options:

בנוסף: בגלל שלשנינו, גם גיתם וגם ללהב יש ניסיון מצטבר של מעל 6 שנות פיתוח, החלטנו להראות את יכולותינו והוספנו 2 בונסים ששווה להסתכל עליהם:

א. Frontend שלם, צד לקוח עם GUI, מסך התחברות, צ'אט בזמן אמת נוטיפיקציות ועוד



Gitam And Lahav's Chat

● Connected as gitam

Disconnect

To send a message, use format
TARGET_USERNAME:your message here
Type 'list' to see online users, 'exit' to disconnect
Type 'everyone:<message>' to send a message to everyone
08:48 PM

SYSTEM: lahav has joined the chat

lahav
Hey gitam, whats up?
08:48 PM

You → lahav
Hey Lahav, i am great man! where are you at ?
08:49 PM

lahav
I am studying for Probability, my probability to pass is sitting on 0 :(
08:49 PM

Online Users

lahav	Online
-------	--------

lahav



Send

בנוס שני - יצירת "דמו" - בעצם, מין תכנית שאפשר להפעיל, ולראות לבד איך בוטים מתחברים לציאט, מדברים אחד עם השני (עם סקריפט מוכן), ויוצאים.

```
Starting server...
Server listening on 127.0.0.1:10000
Server started!

Connecting clients...

Client connected: ('127.0.0.1', 65382)
Active clients: 2
Client ('127.0.0.1', 65382) registered as: Alice
Active users: ['Alice']
[Alice] Connected!
[Alice] Received: To send a message, use format: TARGET_USERNAME:your message here
Type 'list' to see online users, 'exit' to disconnect
Type 'everyone:<message>' to send a message to everyone
Client connected: ('127.0.0.1', 65383)
Active clients: 4
Client ('127.0.0.1', 65383) registered as: Bob
Active users: ['Alice', 'Bob']
[Bob] Connected!
[Bob] Received: To send a message, use format: TARGET_USERNAME:your message here
Type 'list' to see online users, 'exit' to disconnect
Type 'everyone:<message>' to send a message to everyone
Client connected: ('127.0.0.1', 65384)
Active clients: 6
Client ('127.0.0.1', 65384) registered as: Charlie
Active users: ['Alice', 'Bob', 'Charlie']
[Charlie] Connected!
[Charlie] Received: To send a message, use format: TARGET_USERNAME:your message here
Type 'list' to see online users, 'exit' to disconnect
Type 'everyone:<message>' to send a message to everyone
Client connected: ('127.0.0.1', 65385)
Active clients: 8
Client ('127.0.0.1', 65385) registered as: Diana
Active users: ['Alice', 'Bob', 'Charlie', 'Diana']
[Diana] Connected!
[Diana] Received: To send a message, use format: TARGET_USERNAME:your message here
Type 'list' to see online users, 'exit' to disconnect
Type 'everyone:<message>' to send a message to everyone

Starting conversation...

[Alice] → Bob: Hey Bob, how's your day going?
Message from Alice (('127.0.0.1', 65382)): Bob: Hey Bob, how's your day going?
Forwarded message from Alice to Bob
[Bob] Received: Alice: Hey Bob, how's your day going?
[Bob] → Alice: Hi Alice! It's going well, thanks for asking. How about you?
Message from Bob (('127.0.0.1', 65383)): Alice: Hi Alice! It's going well, thanks for asking. How about you?
Forwarded message from Bob to Alice
[Alice] Received: Bob: Hi Alice! It's going well, thanks for asking. How about you?
[Alice] → Bob: I'm doing great, thanks!
Message from Alice (('127.0.0.1', 65382)): Bob: I'm doing great, thanks!
Forwarded message from Alice to Bob
[Bob] Received: Alice: I'm doing great, thanks!
[Alice] → Charlie: Charlie, did you finish that project we discussed?
Message from Alice (('127.0.0.1', 65382)): Charlie: Charlie, did you finish that project we discussed?
Forwarded message from Alice to Charlie
[Charlie] Received: Alice: Charlie, did you finish that project we discussed?
[Charlie] → Alice: Yes, I finished it yesterday. Thanks for checking!
Message from Charlie (('127.0.0.1', 65384)): Alice: Yes, I finished it yesterday. Thanks for checking!
Forwarded message from Charlie to Alice
```